

1. Mérföldkő: Követelményelemzés és Tervezés

1. Megvalósíthatósági terv (Feasibility Study)

A projekt egy közepes méretű egyetemi féléves feladatnak felel meg.

- **Humán erőforrások:** A projekt megvalósításához egy szoftverfejlesztő (diák) munkája szükséges (kb. 35-45 munkaóra a tervezéstől a tesztelésig).
- **Hardver erőforrások:** Átlagos teljesítményű személyi számítógép a fejlesztői környezet és a lokális adatbázis futtatásához.
- **Szoftver erőforrások:** Fejlesztőkörnyezet (Visual Studio, C# nyelv), lokális adatbáziskezelő (SQLite), verziókövető (GitHub), Webes keretrendszer.
- **Üzemeltetés és Karbantartás:** Külön üzemeltetőt nem igényel. Rendszeres (automatizált) adatbázis-mentés beállítása, illetve az esetleges szoftverhibák (bugok) utólagos javítása szükséges.

2. Funkcionális specifikáció

A szoftver által biztosított funkciók (amelyek 1:1-ben leképeződnek a Use-Case diagramon):

- **Bejelentkezés:** A könyvtáros felhasználónévvel és jelszóval lép be. 3 hibás próbálkozás után a program kilép.
- **Könyvek kezelése (CRUD):** Új könyv felvétele (metaadatok és kölcsönözhetőség), módosítása, törlése. Törölni csak ki nem kölcsönzött könyvet lehet (logikai törlés). Több példány is kezelhető.
- **Könyvek keresése és listázása:** Keresés (szerző, cím, ISBN alapján). Több példány esetén a példányszám megjelenítése egy tételként.
- **Tagok kezelése (CRUD):** Új tag felvétele (név, lakcím, típus, elérhetőség), adatainak módosítása, törlése. A rendszer 4 tagtípust különböztet meg.
- **Tagok keresése és listázása:** Keresés név vagy lakcím alapján.
- **Kölcsönzés indítása:** Könyv kiadása tagnak. A rendszer ellenőrzi a könyv elérhetőségét, a tag limitjét, és a tagtípus alapján generálja a lejárat dátumot.
- **Visszavétel és Késés jelzése:** Könyv visszavétele. A rendszer automatikusan jelzi az esetleges késést.
- **Tag aktív kölcsönzéseinek listázása:** Adott tag kint lévő és visszahozott könyveinek megtekintése.

3. Nem funkcionális specifikáció

- **Fejlesztési követelmény:** A szoftver szigorúan objektumorientált (OOP) felépítésű. A könyvtári tagtípusok implementálása öröklődéssel (polimorfizmus) történik.
- **Környezeti követelmény:** Rendelkeznie kell grafikus (GUI) vagy webes felhasználói felülettel. Az adatokat relációs adatbázisban (SQLite) tároljuk.
- **Biztonság:** A rendszer funkcióihoz csak sikeres autentikáció után lehet hozzáférni.

4. Használati esetek (Use Case) és Diagram

A rendszernek egyetlen aktív aktora van: a Könyvtáros. A diagram kizárólag a funkcionális specifikációban meghatározott szolgáltatásokat és azok viszonyait (include, extend) mutatja be.

```
usecaseDiagram
```

```
    actor "Könyvtáros" as Lib
```

```
    package "Könyvtári Adminisztrációs Rendszer" {
```

```
        usecase "Bejelentkezés" as Login
```

```
        usecase "Könyvek kezelése (CRUD)" as ManageBooks
```

```
        usecase "Könyvek keresése és listázása" as SearchBooks
```

```
        usecase "Tagok kezelése (CRUD)" as ManageMembers
```

```
        usecase "Tagok keresése és listázása" as SearchMembers
```

```
        usecase "Kölcsönzés indítása" as Borrow
```

```
        usecase "Limit és Jogosultság ellenőrzése" as CheckLimit
```

```
        usecase "Visszavétel" as Return
```

```
        usecase "Késés jelzése" as Overdue
```

```
        usecase "Tag aktív kölcsönzéseinek listázása" as ListBorrows
```

```
    }
```

```
    Lib --> Login
```

```
    Lib --> ManageBooks
```

```
    Lib --> ManageMembers
```

```
    Lib --> Borrow
```

```
    Lib --> Return
```

ManageBooks <.. SearchBooks : <<extend>>

ManageMembers <.. SearchMembers : <<extend>>

Borrow ..> CheckLimit : <<include>>

Return ..> ListBorrows : <<include>>

Return <.. Overdue : <<extend>>

(Megjegyzés: A "Bejelentkezés" minden más funkció előfeltétele, de az átláthatóság érdekében nem kötöttem be az összes use case-hez).

5. Felhasználói történetek (User Stories) és Tesztesetek

A történetek alapján elvégezhető az alkalmazás manuális tesztelése.

1. Történet (Normál működés): Kölcsönzés indítása

- **Cél:** Mint könyvtáros, szeretnék egy könyvet kikölcsönözni egy tagnak, hogy az hazavihesse.
- **Given (Feltétel):** A kiválasztott tag egy "Egyetemi hallgató", akinek 4 aktív kölcsönzése van (nem érte el a max 5-öt), és a könyv státusza "Szabad".
- **When (Esemény):** Rögzítem a kölcsönzést a felületen.
- **Then (Eredmény):** A rendszer elmenti a tranzakciót, levon egy példányt a szabad készletből, és a visszahozatal dátumát pontosan 2 hónappal későbbre állítja.

2. Történet (Határeset/Hiba): Kölcsönzési limit túllépése

- **Cél:** Mint könyvtáros, biztosítani akarom, hogy senki ne vigyen el több könyvet a megengedettnél.
- **Given (Feltétel):** A tag egy "Mindenki más" kategóriájú felhasználó, akinél már 2 könyv van kint (elérte a limitet).
- **When (Esemény):** Megpróbálok egy 3. könyvet hozzárendelni.
- **Then (Eredmény):** A rendszer megtagadja a mentést, és jól látható hibaüzenetet dob: "A tag elérte a maximális kölcsönzési limitet (2 db)."

3. Történet (Határeset/Hiba): Hibás bejelentkezés kezelése

- **Cél:** Mint könyvtáros, szeretném, hogy a rendszer védve legyen az illetéktelenektől.
- **Given (Feltétel):** A program bejelentkezési képernyője aktív.
- **When (Esemény):** Egymás után háromszor hibás jelszót adok meg.
- **Then (Eredmény):** A harmadik rontott kísérlet után a rendszer hibaüzenetet jelenít meg és a program automatikusan leáll.

6. Rendszerarchitektúra és Modellek

6.1. Komponens diagram

A rendszer logikai és fizikai rétegeinek (Architektúra) elkülönítése:

```
componentDiagram
```

```
package "Prezentációs réteg" {  
    [Felhasználói Felület (UI)]  
}  
  
package "Üzleti logika réteg (BLL)" {  
    [Hitelesítési modul]  
    [Kölcsönzés Menedzser]  
    [Katalógus Menedzser]  
}  
  
package "Adatelérési réteg (DAL)" {  
    [Adatbázis Kliens]  
}  
  
database "SQLite Adatbázis" {  
    [Adattáblák]  
}
```

```
[Felhasználói Felület (UI)] --> [Hitelesítési modul]
[Felhasználói Felület (UI)] --> [Kölcsönzés Menedzser]
[Felhasználói Felület (UI)] --> [Katalógus Menedzser]

[Hitelesítési modul] --> [Adatbázis Kliens]
[Kölcsönzés Menedzser] --> [Adatbázis Kliens]
[Katalógus Menedzser] --> [Adatbázis Kliens]

[Adatbázis Kliens] --> [Adattáblák]
```

6.2. Osztálydiagram (Class Diagram)

A diagram az OOP tervezés alapjait, a tagok közötti öröklődést, az osztályok felelősségét és publikus interfészeit mutatja (a privát segéd metódusokat kihagytam a jobb áttekinthetőségért).

```
classDiagram
    class Konyvtaros {
        -String felhasznalonev
        -String jelszoHash
        +bejelentkezés(user, pass) bool
    }
    class Konyv {
        -String isbn
        -String szerzo
        -String cim
        +getDetails() String
    }
    class Peldany {
        -int peldanyAzonosito
        -String allapot
        +isElheto() bool
    }
```

```
}
```

```
class Tag {  
    <<abstract>>  
    -int tagAzonosito  
    -String nev  
    -String lakcim  
    +getMaxKonyvSzam()* int  
    +getKolcsonzesiIdoNapban()* int  
    +getAktivKolcsonzesekSzama() int  
}
```

```
class EgyetemiHallgato {  
    +getMaxKonyvSzam() int %% Return 5  
    +getKolcsonzesiIdoNapban() int %% Return 60  
}
```

```
class EgyetemiOktato {  
    +getMaxKonyvSzam() int %% Return 9999  
    +getKolcsonzesiIdoNapban() int %% Return 365  
}
```

```
class KulsosPolgar {  
    +getMaxKonyvSzam() int %% Return 4  
    +getKolcsonzesiIdoNapban() int %% Return 30  
}
```

```
class EgyebTag {  
    +getMaxKonyvSzam() int %% Return 2  
    +getKolcsonzesiIdoNapban() int %% Return 14  
}
```

```
class Kolcsonzes {
```

```

    -Date kiadasDatuma

    -Date hataridoDatuma

    -Date visszahozatalDatuma

    +calculateHatarido(Tag tag)

    +isKesesbenVan() bool
}

Konyv "1" *-- "1..*" Peldany : tartalmaz
Tag <|-- EgyetemiHallgato
Tag <|-- EgyetemiOktato
Tag <|-- KulsosPolgar
Tag <|-- EgyebTag
Tag "1" --> "*" Kolcsonzes : indit
Peldany "1" --> "*" Kolcsonzes : érintett

```

6.3. Adatbázis séma diagram (ER Diagram)

Az adatok tárolásához a Tagok esetében "Single Table Inheritance" modellt alkalmazunk egy dedikált type (típus) mezővel.

```

erDiagram
    USERS {
        int id PK
        string username
        string password_hash
    }
    BOOKS {
        string isbn PK
        string title
        string author
        boolean is_deleted
    }
    BOOK_COPIES {
        int copy_id PK
        string isbn FK
    }

```

```

        string status
    }

MEMBERS {

    int member_id PK

    string name

    string type

    string address

    boolean is_deleted
}

LOANS {

    int loan_id PK

    int copy_id FK

    int member_id FK

    date start_date

    date due_date

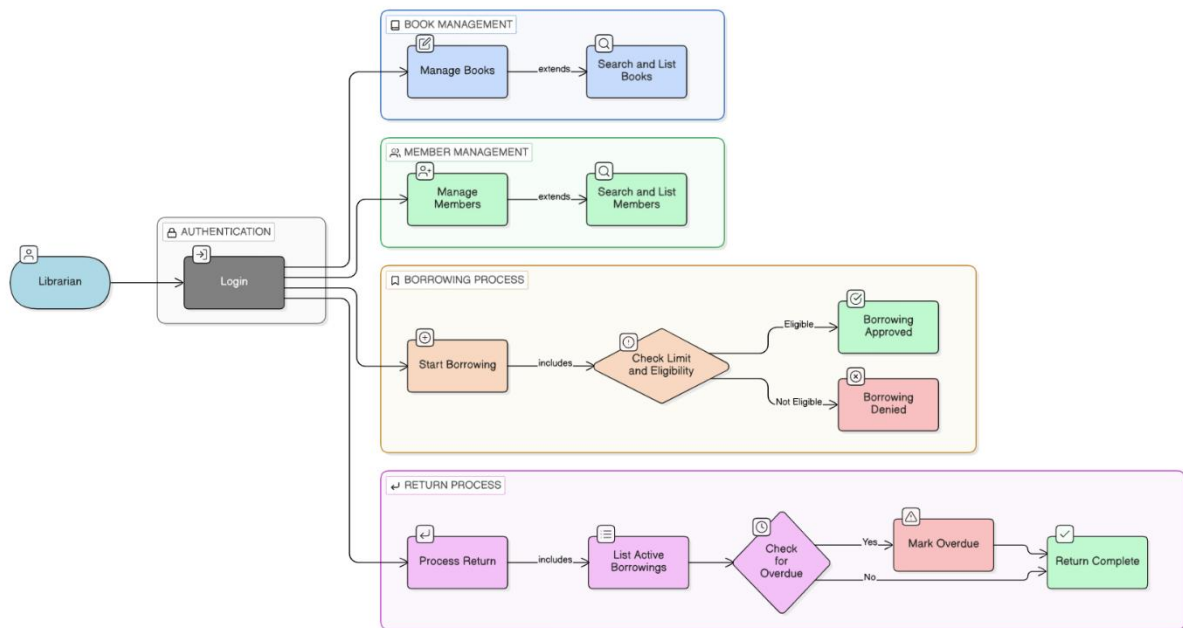
    date return_date
}


BOOKS ||--|{ BOOK_COPIES : "rendelkezik"
MEMBERS ||--|{ LOANS : "kölcsönöz"
BOOK_COPIES ||--o{ LOANS : "szerepel benne"

```

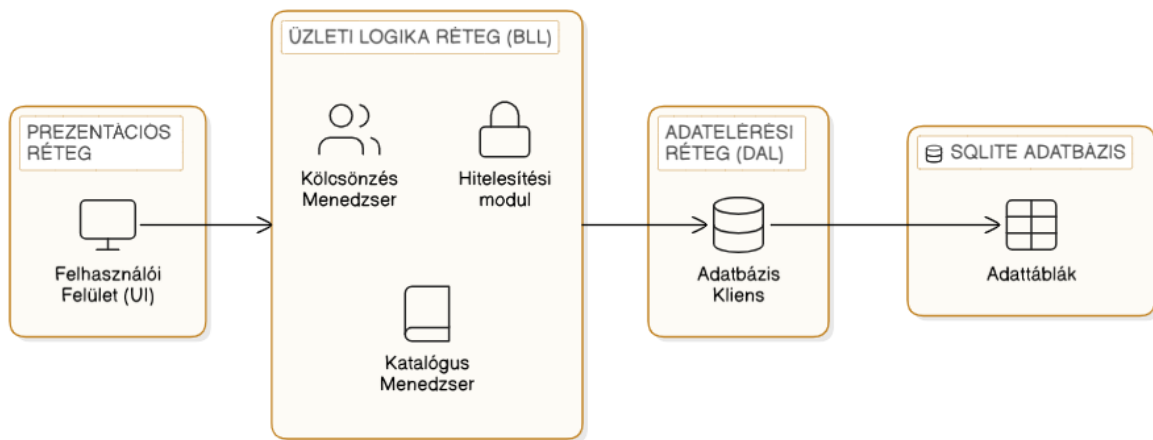
7. Felhasználói felület terv (Wireframes)

Use Case Diagram:



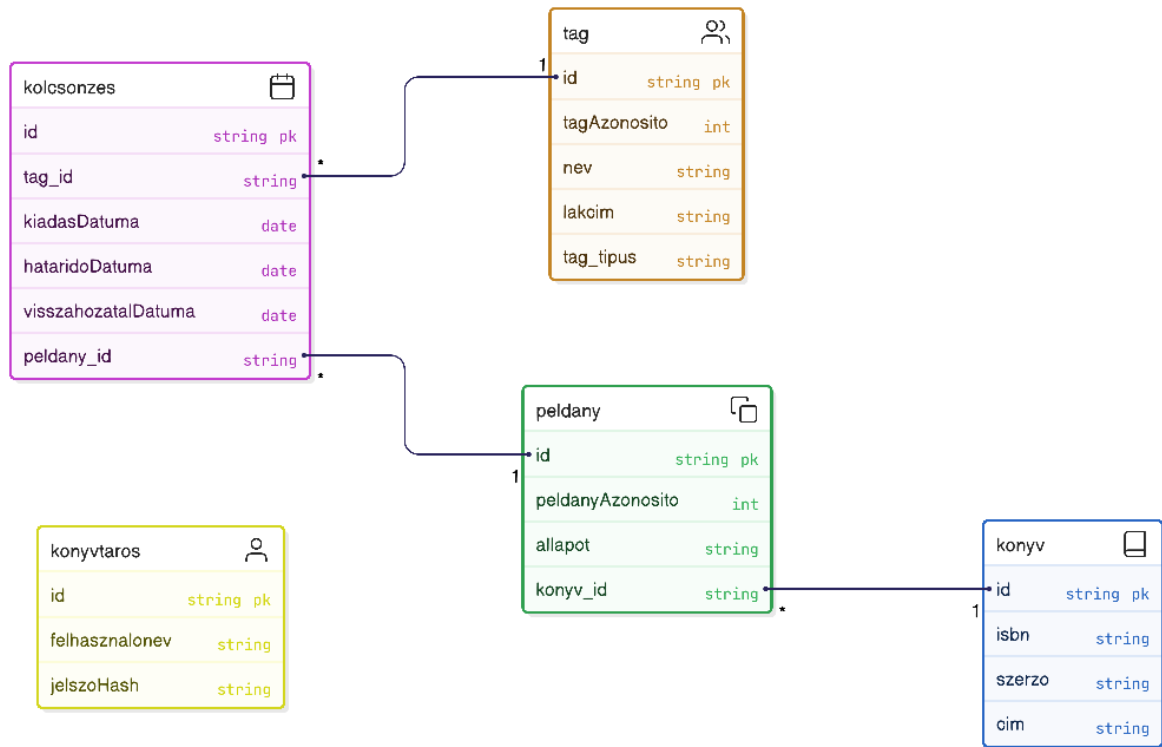
eraser

Component Diagram:

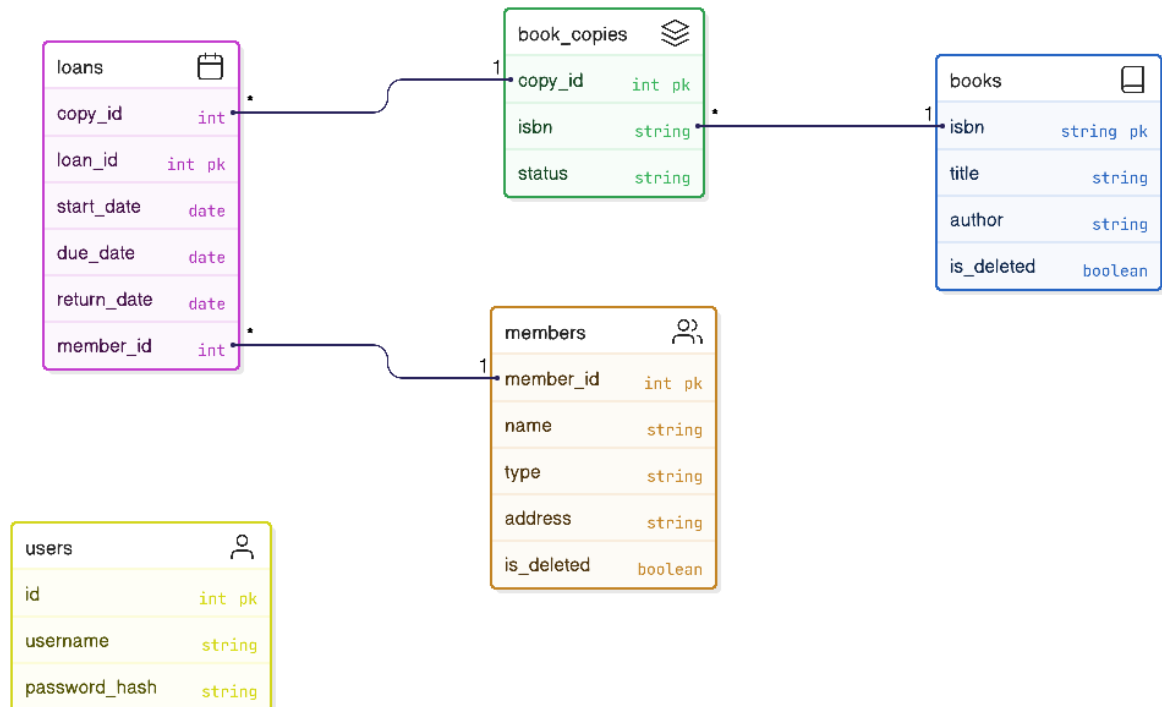


eraser

Class Diagram:



ER Diagram:



Főképernyő navigáció:

[Fejléc: Könyvtári Adminisztrációs Rendszer - Bejelentkezve: admin]

[Oldalsáv Menü]

- Könyvek kezelése
- Tagok kezelése
- Kölcsönzés indítása
- Visszavétel
(Figyelmeztetés!)
- Keresés és Listázás
- Kilépés

[Fő munkaterület]

Üdvözlöm a rendszerben!

Jelenleg aktív kölcsönzések: 42 db

Lejárt határidejű kölcsönzések: 3 db

Kölcsönzés indítása Képernyő:

[Kölcsönzés rögzítése]

Tag kiválasztása: [Keresőmező: Név vagy ID] -> (Eredmény: John Doe - Egyetemi Hallgató)

Aktív könyvek a tagnál: 2/5 db. (Jogosult kölcsönzésre)

Könyv példány beolvasása/kiválasztása: [Keresőmező: ISBN vagy Példány ID]

Kiválasztott könyv: Gárdonyi Géza - Egri csillagok (Szabad)

[Kölcsönzés Rögzítése (Gomb)]

A rendszer automatikusan számolt lejáratí ideje: 2026.05.14.